

Exploring_Generative_AI_Libraries

January 16, 2026

1 Exploring Generative AI Libraries

Estimated time needed: **60** minutes

1.1 Table of Contents

Objectives

What is generative AI?

Real-world impact of generative AI

Art and creativity

Natural language processing (NLP)

Computer vision

Virtual avatars

Text generation before transformers

N-gram language models

Recurrent neural networks (RNN)

Long short-term memory (LSTM) and gated recurrent units (GRUs)

Seq2seq models with attention

Transformers

Implementat

Step 1: Installing libraries

<a href="#Step-2:-Importing-the-required-tools-from-the-transformers-libra

Step 2: Importing the required tools from the transformers library

1.2 Objectives

After completing this lab, you will be able to:

- Gain an understanding of generative AI and its impact across various domains.
- Familiarize yourself with different types of models in generative AI.
- Acquire the skills to build and interact with a chatbot using transformers.

1.3 What is generative AI?

Imagine presenting a computer with a vast array of paintings. After analyzing them, it tries to craft a unique painting of its own. This capability is termed generative AI. Essentially, the computer derives inspiration from the provided content and uses it to create something new.

1.4 Real-world impact of generative AI

Generative AI is transforming multiple industries. Its applications span:

1.4.1 1. Art and creativity

- Generative art: Artists employing generative AI algorithms can create stunning artworks by learning from existing masterpieces and producing unique pieces inspired by them. These AI-generated artworks have gained recognition in the art world.
- Music Composition: Projects in the realm of generative AI have been employed to compose music. They learn from a vast data set of musical compositions and can generate original pieces in various styles, from classical to jazz, revolutionizing the music industry.

1.4.2 2. Natural language processing (NLP)

- Content generation: Tools like generative pre-trained transformer (GPT) have demonstrated their ability to generate coherent and context-aware text. They can assist content creators by generating articles, stories, or marketing copy, making them valuable tools in content creation.
- Chatbots and virtual assistants: Generative AI powers many of today's chatbots and virtual assistants. These AI-driven conversational agents understand and generate human-like responses, enhancing user experiences.
- Code Writing: Generative AI models can also produce code snippets based on descriptions or requirements, streamlining software development.

1.4.3 3. Computer vision

- Image synthesis: Models like data analysis learning with language model for generation and exploration, frequently known as DALL-E, can generate images from textual descriptions. This technology finds applications in graphic design, advertising, and creating visual content for marketing.
- Deepfake detection: With the advancement in generative AI techniques, the generation of deep fake content is also on the rise. Consequently, generative AI now plays a role in developing tools and techniques to detect and combat the spread of misinformation through manipulated videos.

1.4.4 4. Virtual avatars

- Entertainment: Generative AI is utilized to craft virtual avatars for gaming and entertainment. These avatars mimic human expressions and emotions, bolstering user engagement in virtual environments.
- Marketing: Virtual influencers, propelled by generative AI, are on the rise in digital marketing. Brands are harnessing these virtual personas to endorse their products and services.

1.5 Neural structures behind generative AI

Before we had the powerful transformers, which are like super-fast readers and understand lots of words at once, there were other methods used for making computers generate text. These methods were like the building blocks that led to the amazing capabilities we have today.

1.6 Large language models (LLMs)

Large language models are like supercharged brains. They are massive computer programs with lots of “neurons” that learn from huge amounts of text. These models are trained to do tasks like understanding and generating text, and they’re used in many applications. However, there’s a limitation: these models are not very good at understanding the bigger context or the meaning of words. They work well for simple predictions but struggle with more complex text.

1.7 Text generation before transformers

1.7.1 1. N-gram language models

N-gram models are like language detectives. They predict what words come next in a sentence based on the words that came before. For example, if you say “The sky is,” these models guess that the next word might be “blue.”

1.7.2 2. Recurrent neural networks (RNN)

Recurrent neural networks (RNNs) are specially designed to handle sequential data, making them a powerful tool for applications like language modeling and time series forecasting. The essence of their design lies in maintaining a ‘memory’ or ‘hidden state’ throughout the sequence by employing loops. This enables RNNs to recognize and capture the temporal dependencies inherent in sequential data. - Hidden state: Often referred to as the network’s ‘memory’, the hidden state is a dynamic storage of information about previous sequence inputs. With each new input, this hidden state is updated, factoring in both the new input and its previous value. - Temporal dependency: Loops in RNNs enable information transfer across sequence steps.

Image Source

Illustration of RNN’s operation: Consider a simple sequence, such as the sentence: “I love RNNs”. The RNN interprets this sentence word by word. Beginning with the word “I”, the RNN ingests it, generates an output, and updates its hidden state. Moving on to “love”, the RNN processes it alongside the updated hidden state which already holds insights about the word “I”. The hidden state is updated again post this. This pattern of processing and updating continues until the last word. By the end of the sequence, the hidden state ideally encapsulates insights from the entire sentence.

1.7.3 3. Long short-term memory (LSTM) and gated recurrent units (GRUs)

Long short-term memory (LSTM) and gated recurrent units (GRUs) are advanced variations of recurrent neural networks (RNNs), designed to address the limitations of traditional RNNs and enhance their ability to model sequential data effectively. They processed sequences one element at a time and maintained an internal state to remember past elements. While they were effective for a variety of tasks, they struggled with long sequences and long-term dependencies.

1.7.4 4. Seq2seq models with attention

- Sequence-to-sequence (seq2seq) models, often built with RNNs or LSTMs, were designed to handle tasks like translation where an input sequence is transformed into an output sequence.
- The attention mechanism was introduced to allow the model to “focus” on relevant parts of the input sequence when generating the output, significantly improving performance on tasks like machine translation.

While these methods provided significant advancements in text generation tasks, the introduction of transformers led to a paradigm shift. Transformers, with their self-attention mechanism, proved to be highly efficient at capturing contextual information across long sequences, setting new benchmarks in various NLP tasks.

1.8 Transformers

Proposed in a paper titled “Attention Is All You Need” by Vaswani et al. in 2017, the transformer architecture replaced sequential processing with parallel processing. The key component behind its success? The attention mechanism, more precisely, self-attention.

Key steps include:

- Tokenization: The first step is breaking down a sentence into tokens (words or subwords).
- Embedding: Each token is represented as a vector, capturing its meaning.
- Self-attention: The model computes scores determining the importance of every other word for a particular word in the sequence. These scores are used to weight the input tokens and produce a new representation of the sequence. For instance, in the sentence “He gave her a gift because she’d helped him”, understanding who “her” refers to requires the model to pay attention to other words in the sentence. The transformer does this for every word, considering the entire context, which is particularly powerful for understanding meaning.
- Feed-forward neural networks: After attention, each position is passed through a feed-forward network separately.
- Output sequence: The model produces an output sequence, which can be used for various tasks, like classification, translation, or text generation.
- Layering: Importantly, transformers are deep models with multiple layers of attention and feed-forward networks, allowing them to learn complex patterns.

The architecture’s flexibility has allowed transformers to be used beyond NLP, finding applications in image and video processing too. In NLP, transformer-based models like BERT, GPT, and their variants have set state-of-the-art results in various tasks, from text classification to translation.

1.8.1 Implementation: Building a simple chatbot with transformers

Now, you will build a simple chatbot using `transformers` library from Hugging Face, which is an open-source natural language processing (NLP) toolkit with many useful features. ##### Step 1: Installing libraries

```
[1]: %pip install torch==2.8.0+cpu torchvision==0.23.0+cpu torchaudio==2.8.0+cpu \
      ↪torchtext==0.18.0+cpu \
      --index-url https://download.pytorch.org/whl/cpu
      %pip install transformers==4.42.1 tensorflow sentencepiece numpy==1.26
```

```
Looking in indexes: https://download.pytorch.org/whl/cpu
Collecting torch==2.8.0+cpu
  Downloading https://download.pytorch.org/whl/cpu/torch-2.8.0%2Bcpu-
cp312-cp312-manylinux_2_28_x86_64.whl.metadata (29 kB)
Collecting torchvision==0.23.0+cpu
  Downloading https://download.pytorch.org/whl/cpu/torchvision-0.23.0%2Bcpu-
cp312-cp312-manylinux_2_28_x86_64.whl.metadata (6.1 kB)
Collecting torchaudio==2.8.0+cpu
  Downloading https://download.pytorch.org/whl/cpu/torchaudio-2.8.0%2Bcpu-
cp312-cp312-manylinux_2_28_x86_64.whl.metadata (7.2 kB)
Collecting torchtext==0.18.0+cpu
  Downloading https://download.pytorch.org/whl/cpu/torchtext-0.18.0%2Bcpu-
cp312-cp312-linux_x86_64.whl (2.0 MB)
      2.0/2.0 MB
21.9 MB/s eta 0:00:00
Collecting filelock (from torch==2.8.0+cpu)
  Downloading filelock-3.20.0-py3-none-any.whl.metadata (2.1 kB)
Requirement already satisfied: typing-extensions>=4.10.0 in
/opt/conda/lib/python3.12/site-packages (from torch==2.8.0+cpu) (4.12.2)
Requirement already satisfied: setuptools in /opt/conda/lib/python3.12/site-
packages (from torch==2.8.0+cpu) (75.8.0)
Collecting sympy>=1.13.3 (from torch==2.8.0+cpu)
  Downloading sympy-1.14.0-py3-none-any.whl.metadata (12 kB)
Collecting networkx (from torch==2.8.0+cpu)
  Downloading networkx-3.6.1-py3-none-any.whl.metadata (6.8 kB)
Requirement already satisfied: Jinja2 in /opt/conda/lib/python3.12/site-packages
(from torch==2.8.0+cpu) (3.1.5)
Collecting fsspec (from torch==2.8.0+cpu)
  Downloading fsspec-2025.12.0-py3-none-any.whl.metadata (10 kB)
Collecting numpy (from torchvision==0.23.0+cpu)
  Downloading
numpy-2.3.5-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata
(62 kB)
Collecting pillow!=8.3.*,>=5.3.0 (from torchvision==0.23.0+cpu)
  Downloading pillow-12.0.0-cp312-cp312-
manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (8.8 kB)
Requirement already satisfied: tqdm in /opt/conda/lib/python3.12/site-packages
(from torchtext==0.18.0+cpu) (4.67.1)
Requirement already satisfied: requests in /opt/conda/lib/python3.12/site-
packages (from torchtext==0.18.0+cpu) (2.32.3)
Collecting mpmath<1.4,>=1.1.0 (from sympy>=1.13.3->torch==2.8.0+cpu)
  Downloading mpmath-1.3.0-py3-none-any.whl.metadata (8.6 kB)
Requirement already satisfied: MarkupSafe>=2.0 in
```

```

/opt/conda/lib/python3.12/site-packages (from jinja2->torch==2.8.0+cpu) (3.0.2)
Requirement already satisfied: charset_normalizer<4,>=2 in
/opt/conda/lib/python3.12/site-packages (from requests->torchtext==0.18.0+cpu)
(3.4.1)
Requirement already satisfied: idna<4,>=2.5 in /opt/conda/lib/python3.12/site-
packages (from requests->torchtext==0.18.0+cpu) (3.10)
Requirement already satisfied: urllib3<3,>=1.21.1 in
/opt/conda/lib/python3.12/site-packages (from requests->torchtext==0.18.0+cpu)
(2.3.0)
Requirement already satisfied: certifi>=2017.4.17 in
/opt/conda/lib/python3.12/site-packages (from requests->torchtext==0.18.0+cpu)
(2024.12.14)
Downloading https://download.pytorch.org/whl/cpu/torch-2.8.0%2Bcpu-
cp312-cp312-manylinux_2_28_x86_64.whl (183.9 MB)
183.9/183.9 MB
13.5 MB/s eta 0:00:0000:0100:01
Downloading https://download.pytorch.org/whl/cpu/torchvision-0.23.0%2Bcpu-
cp312-cp312-manylinux_2_28_x86_64.whl (1.9 MB)
1.9/1.9 MB
151.1 kB/s eta 0:00:00a 0:00:01
Downloading https://download.pytorch.org/whl/cpu/torchaudio-2.8.0%2Bcpu-
cp312-cp312-manylinux_2_28_x86_64.whl (1.8 MB)
1.8/1.8 MB
174.9 kB/s eta 0:00:00a 0:00:01
Downloading
pillow-12.0.0-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl (7.0
MB)
7.0/7.0 MB
579.4 kB/s eta 0:00:00a 0:00:01
Downloading sympy-1.14.0-py3-none-any.whl (6.3 MB)
6.3/6.3 MB
386.6 kB/s eta 0:00:0000:01
Downloading filelock-3.20.0-py3-none-any.whl (16 kB)
Downloading fsspec-2025.12.0-py3-none-any.whl (201 kB)
Downloading networkx-3.6.1-py3-none-any.whl (2.1 MB)
2.1/2.1 MB
54.0 MB/s eta 0:00:00
Downloading
numpy-2.3.5-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl (16.6
MB)
16.6/16.6 MB
37.0 MB/s eta 0:00:00:00:01
Downloading mpmath-1.3.0-py3-none-any.whl (536 kB)
536.2/536.2 kB
9.0 MB/s eta 0:00:00
Installing collected packages: mpmath, sympy, pillow, numpy, networkx, fsspec,
filelock, torch, torchvision, torchtext, torchaudio
Successfully installed filelock-3.20.0 fsspec-2025.12.0 mpmath-1.3.0

```

```

networkx-3.6.1 numpy-2.3.5 pillow-12.0.0 sympy-1.14.0 torch-2.8.0+cpu
torchaudio-2.8.0+cpu torchtext-0.18.0+cpu torchvision-0.23.0+cpu
Note: you may need to restart the kernel to use updated packages.
Collecting transformers==4.42.1
  Downloading transformers-4.42.1-py3-none-any.whl.metadata (43 kB)
Collecting tensorflow
  Downloading tensorflow-2.20.0-cp312-cp312-
manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (4.5 kB)
Collecting sentencepiece
  Downloading sentencepiece-0.2.1-cp312-cp312-
manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (10 kB)
Collecting numpy==1.26
  Downloading
numpy-1.26.0-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata
(58 kB)
Requirement already satisfied: filelock in /opt/conda/lib/python3.12/site-
packages (from transformers==4.42.1) (3.20.0)
Collecting huggingface-hub<1.0,>=0.23.2 (from transformers==4.42.1)
  Downloading huggingface_hub-0.36.0-py3-none-any.whl.metadata (14 kB)
Requirement already satisfied: packaging>=20.0 in
/opt/conda/lib/python3.12/site-packages (from transformers==4.42.1) (24.2)
Requirement already satisfied: pyyaml>=5.1 in /opt/conda/lib/python3.12/site-
packages (from transformers==4.42.1) (6.0.2)
Collecting regex!=2019.12.17 (from transformers==4.42.1)
  Downloading regex-2025.11.3-cp312-cp312-
manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl.metadata
(40 kB)
Requirement already satisfied: requests in /opt/conda/lib/python3.12/site-
packages (from transformers==4.42.1) (2.32.3)
Collecting tokenizers<0.20,>=0.19 (from transformers==4.42.1)
  Downloading tokenizers-0.19.1-cp312-cp312-
manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (6.7 kB)
Collecting safetensors>=0.4.1 (from transformers==4.42.1)
  Downloading safetensors-0.7.0-cp38-abi3-
manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (4.1 kB)
Requirement already satisfied: tqdm>=4.27 in /opt/conda/lib/python3.12/site-
packages (from transformers==4.42.1) (4.67.1)
Collecting absl-py>=1.0.0 (from tensorflow)
  Downloading absl_py-2.3.1-py3-none-any.whl.metadata (3.3 kB)
Collecting astunparse>=1.6.0 (from tensorflow)
  Downloading astunparse-1.6.3-py2.py3-none-any.whl.metadata (4.4 kB)
Collecting flatbuffers>=24.3.25 (from tensorflow)
  Downloading flatbuffers-25.12.19-py2.py3-none-any.whl.metadata (1.0 kB)
Collecting gast!=0.5.0,!0.5.1,!0.5.2,>=0.2.1 (from tensorflow)
  Downloading gast-0.7.0-py3-none-any.whl.metadata (1.5 kB)
Collecting google_pasta>=0.1.1 (from tensorflow)
  Downloading google_pasta-0.2.0-py3-none-any.whl.metadata (814 bytes)
Collecting libclang>=13.0.0 (from tensorflow)

```

Downloading libclang-18.1.1-py2.py3-none-manylinux2010_x86_64.whl.metadata (5.2 kB)
 Collecting opt_einsum>=2.3.2 (from tensorflow)
 Downloading opt_einsum-3.4.0-py3-none-any.whl.metadata (6.3 kB)
 Collecting protobuf>=5.28.0 (from tensorflow)
 Downloading protobuf-6.33.2-cp39-abi3-manylinux2014_x86_64.whl.metadata (593 bytes)
 Requirement already satisfied: setuptools in /opt/conda/lib/python3.12/site-packages (from tensorflow) (75.8.0)
 Requirement already satisfied: six>=1.12.0 in /opt/conda/lib/python3.12/site-packages (from tensorflow) (1.17.0)
 Collecting termcolor>=1.1.0 (from tensorflow)
 Downloading termcolor-3.2.0-py3-none-any.whl.metadata (6.4 kB)
 Requirement already satisfied: typing_extensions>=3.6.6 in /opt/conda/lib/python3.12/site-packages (from tensorflow) (4.12.2)
 Collecting wrapt>=1.11.0 (from tensorflow)
 Downloading wrapt-2.0.1-cp312-cp312-manylinux1_x86_64.manylinux_2_28_x86_64.manylinux_2_5_x86_64.whl.metadata (9.0 kB)
 Collecting grpcio<2.0,>=1.24.3 (from tensorflow)
 Downloading grpcio-1.76.0-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.whl.metadata (3.7 kB)
 Collecting tensorboard~=2.20.0 (from tensorflow)
 Downloading tensorboard-2.20.0-py3-none-any.whl.metadata (1.8 kB)
 Collecting keras>=3.10.0 (from tensorflow)
 Downloading keras-3.13.0-py3-none-any.whl.metadata (6.3 kB)
 Collecting h5py>=3.11.0 (from tensorflow)
 Downloading h5py-3.15.1-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (3.0 kB)
 Collecting ml_dtypes<1.0.0,>=0.5.1 (from tensorflow)
 Downloading ml_dtypes-0.5.4-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (8.9 kB)
 Requirement already satisfied: wheel<1.0,>=0.23.0 in /opt/conda/lib/python3.12/site-packages (from astunparse>=1.6.0->tensorflow) (0.45.1)
 Requirement already satisfied: fsspec>=2023.5.0 in /opt/conda/lib/python3.12/site-packages (from huggingface-hub<1.0,>=0.23.2->transformers==4.42.1) (2025.12.0)
 Collecting hf_xet<2.0.0,>=1.1.3 (from huggingface-hub<1.0,>=0.23.2->transformers==4.42.1)
 Downloading hf_xet-1.2.0-cp37-abi3-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (4.9 kB)
 Collecting rich (from keras>=3.10.0->tensorflow)
 Downloading rich-14.2.0-py3-none-any.whl.metadata (18 kB)
 Collecting namex (from keras>=3.10.0->tensorflow)
 Downloading namex-0.1.0-py3-none-any.whl.metadata (322 bytes)


```

Collecting optree (from keras>=3.10.0->tensorflow)
  Downloading optree-0.18.0-cp312-cp312-
manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (34 kB)
Requirement already satisfied: charset_normalizer<4,>=2 in
/opt/conda/lib/python3.12/site-packages (from requests->transformers==4.42.1)
(3.4.1)
Requirement already satisfied: idna<4,>=2.5 in /opt/conda/lib/python3.12/site-
packages (from requests->transformers==4.42.1) (3.10)
Requirement already satisfied: urllib3<3,>=1.21.1 in
/opt/conda/lib/python3.12/site-packages (from requests->transformers==4.42.1)
(2.3.0)
Requirement already satisfied: certifi>=2017.4.17 in
/opt/conda/lib/python3.12/site-packages (from requests->transformers==4.42.1)
(2024.12.14)
Collecting markdown>=2.6.8 (from tensorboard~=2.20.0->tensorflow)
  Downloading markdown-3.10-py3-none-any.whl.metadata (5.1 kB)
Requirement already satisfied: pillow in /opt/conda/lib/python3.12/site-packages
(from tensorboard~=2.20.0->tensorflow) (12.0.0)
Collecting tensorboard-data-server<0.8.0,>=0.7.0 (from
tensorboard~=2.20.0->tensorflow)
  Downloading tensorboard_data_server-0.7.2-py3-none-
manylinux_2_31_x86_64.whl.metadata (1.1 kB)
Collecting werkzeug>=1.0.1 (from tensorboard~=2.20.0->tensorflow)
  Downloading werkzeug-3.1.4-py3-none-any.whl.metadata (4.0 kB)
Requirement already satisfied: markupsafe>=2.1.1 in
/opt/conda/lib/python3.12/site-packages (from
werkzeug>=1.0.1->tensorboard~=2.20.0->tensorflow) (3.0.2)
Collecting markdown-it-py>=2.2.0 (from rich->keras>=3.10.0->tensorflow)
  Downloading markdown_it_py-4.0.0-py3-none-any.whl.metadata (7.3 kB)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in
/opt/conda/lib/python3.12/site-packages (from rich->keras>=3.10.0->tensorflow)
(2.19.1)
Collecting mdurl~=0.1 (from markdown-it-
py>=2.2.0->rich->keras>=3.10.0->tensorflow)
  Downloading mdurl-0.1.2-py3-none-any.whl.metadata (1.6 kB)
Downloading transformers-4.42.1-py3-none-any.whl (9.3 MB)
9.3/9.3 MB
136.4 MB/s eta 0:00:00
Downloading
numpy-1.26.0-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (17.9
MB)
17.9/17.9 MB
186.6 MB/s eta 0:00:00
Downloading
tensorflow-2.20.0-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl
(620.7 MB)
620.7/620.7 MB
? eta 0:00:00 0:00:0100:01

```

Downloading
sentencepiece-0.2.1-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl
(1.4 MB)
1.4/1.4 MB
24.9 MB/s eta 0:00:00
Downloading absl_py-2.3.1-py3-none-any.whl (135 kB)
Downloading astunparse-1.6.3-py2.py3-none-any.whl (12 kB)
Downloading flatbuffers-25.12.19-py2.py3-none-any.whl (26 kB)
Downloading gast-0.7.0-py3-none-any.whl (22 kB)
Downloading google_pasta-0.2.0-py3-none-any.whl (57 kB)
Downloading
grpcio-1.76.0-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.whl (6.6
MB)
6.6/6.6 MB
38.7 MB/s eta 0:00:00
Downloading
h5py-3.15.1-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl (5.1 MB)
5.1/5.1 MB
33.3 MB/s eta 0:00:00
Downloading huggingface_hub-0.36.0-py3-none-any.whl (566 kB)
566.1/566.1 kB
11.9 MB/s eta 0:00:00
Downloading keras-3.13.0-py3-none-any.whl (1.5 MB)
1.5/1.5 MB
28.0 MB/s eta 0:00:00
Downloading libclang-18.1.1-py2.py3-none-manylinux2010_x86_64.whl (24.5
MB)
24.5/24.5 MB
39.6 MB/s eta 0:00:00:01
Downloading
ml_dtypes-0.5.4-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl (5.0
MB)
5.0/5.0 MB
39.0 MB/s eta 0:00:00
Downloading opt_einsum-3.4.0-py3-none-any.whl (71 kB)
Downloading protobuf-6.33.2-cp39-abi3-manylinux2014_x86_64.whl (323 kB)
Downloading regex-2025.11.3-cp312-cp312-
manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (803 kB)
803.5/803.5 kB
17.6 MB/s eta 0:00:00
Downloading
safetensors-0.7.0-cp38-abi3-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (507
kB)
Downloading tensorboard-2.20.0-py3-none-any.whl (5.5 MB)
5.5/5.5 MB
35.2 MB/s eta 0:00:00
Downloading termcolor-3.2.0-py3-none-any.whl (7.7 kB)
Downloading

```

tokenizers-0.19.1-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl
(3.6 MB)
3.6/3.6 MB
33.2 MB/s eta 0:00:00
Downloading wrapt-2.0.1-cp312-cp312-
manylinux1_x86_64.manylinux_2_28_x86_64.manylinux_2_5_x86_64.whl (121 kB)
Downloading
hf_xet-1.2.0-cp37-abi3-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (3.3 MB)
3.3/3.3 MB
36.9 MB/s eta 0:00:00
Downloading markdown-3.10-py3-none-any.whl (107 kB)
Downloading tensorboard_data_server-0.7.2-py3-none-manylinux_2_31_x86_64.whl
(6.6 MB)
6.6/6.6 MB
48.2 MB/s eta 0:00:00
Downloading werkzeug-3.1.4-py3-none-any.whl (224 kB)
Downloading namex-0.1.0-py3-none-any.whl (5.9 kB)
Downloading
optree-0.18.0-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl (408
kB)
Downloading rich-14.2.0-py3-none-any.whl (243 kB)
Downloading markdown_it_py-4.0.0-py3-none-any.whl (87 kB)
Downloading mdurl-0.1.2-py3-none-any.whl (10.0 kB)
Installing collected packages: namex, libclang, flatbuffers, wrapt, werkzeug,
termcolor, tensorboard-data-server, sentencepiece, safetensors, regex, protobuf,
optree, opt_einsum, numpy, mdurl, markdown, hf-xet, grpcio, google_pasta, gast,
astunparse, absl-py, tensorboard, ml_dtypes, markdown-it-py, huggingface-hub,
h5py, tokenizers, rich, transformers, keras, tensorflow
Attempting uninstall: numpy
Found existing installation: numpy 2.3.5
Uninstalling numpy-2.3.5:
Successfully uninstalled numpy-2.3.5
Successfully installed absl-py-2.3.1 astunparse-1.6.3 flatbuffers-25.12.19
gast-0.7.0 google_pasta-0.2.0 grpcio-1.76.0 h5py-3.15.1 hf-xet-1.2.0
huggingface-hub-0.36.0 keras-3.13.0 libclang-18.1.1 markdown-3.10 markdown-it-
py-4.0.0 mdurl-0.1.2 ml_dtypes-0.5.4 namex-0.1.0 numpy-1.26.0 opt_einsum-3.4.0
optree-0.18.0 protobuf-6.33.2 regex-2025.11.3 rich-14.2.0 safetensors-0.7.0
sentencepiece-0.2.1 tensorboard-2.20.0 tensorboard-data-server-0.7.2
tensorflow-2.20.0 termcolor-3.2.0 tokenizers-0.19.1 transformers-4.42.1
werkzeug-3.1.4 wrapt-2.0.1
Note: you may need to restart the kernel to use updated packages.

```

Step 2: Importing the required tools from the transformers library In the upcoming script, you initiate variables using two invaluable classes from the transformers library: - `model` is an instance of the class `AutoModelForSeq2SeqLM`. This class lets you interact with your chosen language model. - `tokenizer` is an instance of the class `AutoTokenizer`. This class streamlines your input and presents it to the language model in the most efficient manner. It achieves this by

converting your text input into “tokens”, which is the model’s preferred way of interpreting text. You choose “facebook/blenderbot-400M-distill” for this example model because it is freely available under an open-source license and operates at a relatively brisk pace. For a diverse range of models and their capabilities, you can explore the Hugging Face website: [Hugging Face Models](#).

```
[ ]: from transformers import AutoTokenizer, AutoModelForSeq2SeqLM

# Selecting the model. You will be using "facebook/blenderbot-400M-distill" in
↳ this example.
model_name = "facebook/blenderbot-400M-distill"

# Load the model and tokenizer
model = AutoModelForSeq2SeqLM.from_pretrained(model_name)
tokenizer = AutoTokenizer.from_pretrained(model_name)
```

Following the initialization, let’s set up the chat function to enable real-time interaction with the chatbot.

```
[4]: # Define the chat function
def chat_with_bot():
    while True:
        # Get user input
        input_text = input("You: ")

        # Exit conditions
        if input_text.lower() in ["quit", "exit", "bye"]:
            print("Chatbot: Goodbye!")
            break

        # Tokenize input and generate response
        inputs = tokenizer.encode(input_text, return_tensors="pt")
        outputs = model.generate(inputs, max_new_tokens=150)
        response = tokenizer.decode(outputs[0], skip_special_tokens=True).
↳ strip()

        # Display bot's response
        print("Chatbot:", response)

# Start chatting
chat_with_bot()
```

You: bye

Chatbot: Goodbye!

Alright! You have successfully interacted with your chatbot. By providing it with a prompt, the chatbot used the power of the transformers library and the underlying model to generate a response. This exemplifies the prowess of transformer-based models in comprehending and generating human-like text based on a given context. As you continue to engage with it, you will observe its capacity

to simulate a wide range of conversational topics and styles.

Step 3: Trying another language model and comparing the output You can use a different language model, for example the “[flan-t5-base](#)” model from Google, to create a similar chatbot. You can use a chat function similar to the one defined in Step 2 and compare the outputs of both models.

```
[2]: import sentencepiece
      from transformers import AutoTokenizer, AutoModelForSeq2SeqLM

      model_name = "google/flan-t5-base"
      tokenizer = AutoTokenizer.from_pretrained(model_name)
      model = AutoModelForSeq2SeqLM.from_pretrained(model_name)

tokenizer_config.json: 0.00B [00:00, ?B/s]
spiece.model: 0%|          | 0.00/792k [00:00<?, ?B/s]
tokenizer.json: 0.00B [00:00, ?B/s]
special_tokens_map.json: 0.00B [00:00, ?B/s]
config.json: 0.00B [00:00, ?B/s]
model.safetensors: 0%|          | 0.00/990M [00:00<?, ?B/s]
generation_config.json: 0%|          | 0.00/147 [00:00<?, ?B/s]

[3]: ### Let's chat with another bot
      def chat_with_another_bot():
          while True:
              # Get user input
              input_text = input("You: ")

              # Exit conditions
              if input_text.lower() in ["quit", "exit", "bye"]:
                  print("Chatbot: Goodbye!")
                  break

              # Tokenize input and generate response
              inputs = tokenizer.encode(input_text, return_tensors="pt")
              outputs = model.generate(inputs, max_new_tokens=150)
              response = tokenizer.decode(outputs[0], skip_special_tokens=True).
↪strip()

              # Display bot's response
              print("Chatbot:", response)

      # Start chatting
      chat_with_another_bot()
```

You: bye

Chatbot: Goodbye!

There are many language models available in Hugging Face. In the following exercise, you will compare the output for the same input using two different models.

2 Exercise

2.0.1 Create a chatbot using different models from Hugging Face

Create a simple chatbot using the transformers library from Hugging Face(<https://huggingface.co/models>). Run the code using the following model and compare the output. The model is “[google/flan-t5-small](#)”. (Note: Based on the selected model, you may notice differences in the chatbot output. Multiple factors, such as model training and fine-tuning, influence the output.)

```
[3]: # Add code for the exercise here:
def chat_with_flan_t5_small():
    while True:
        # Get user input
        input_text = input("You: ")

        # Exit conditions
        if input_text.lower() in ["quit", "exit", "bye"]:
            print("Chatbot: Goodbye!")
            break

        # Tokenize input and generate response
        inputs = tokenizer.encode(input_text, return_tensors="pt")
        outputs = model.generate(inputs, max_new_tokens=150)
        response = tokenizer.decode(outputs[0], skip_special_tokens=True).
        ↪strip()

        # Display bot's response
        print("Chatbot:", response)

# Start chatting
chat_with_flan_t5_small()
```

You: bye

Chatbot: Goodbye!

Click here for Solution

```
import sentencepiece
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM

model_name = "google/flan-t5-small" #here the model name can be changed as you like.
```

```
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForSeq2SeqLM.from_pretrained(model_name)
```

3 Congratulations! You have completed the lab.

3.1 Authors

[Vicky Kuo](#) is completing her Master's degree in IT at York University with scholarships. Her master's thesis explores the optimization of deep learning algorithms, employing an innovative approach to scrutinize and enhance neural network structures and performance.

© Copyright IBM Corporation. All rights reserved.

{## Change Log}

{|Date (YYYY-MM-DD)|Version|Changed By|Change Description|}
{|2023-09-10|0.1|Vicky Kuo|Initial Lab Created|}

{|-|-|-|-|}